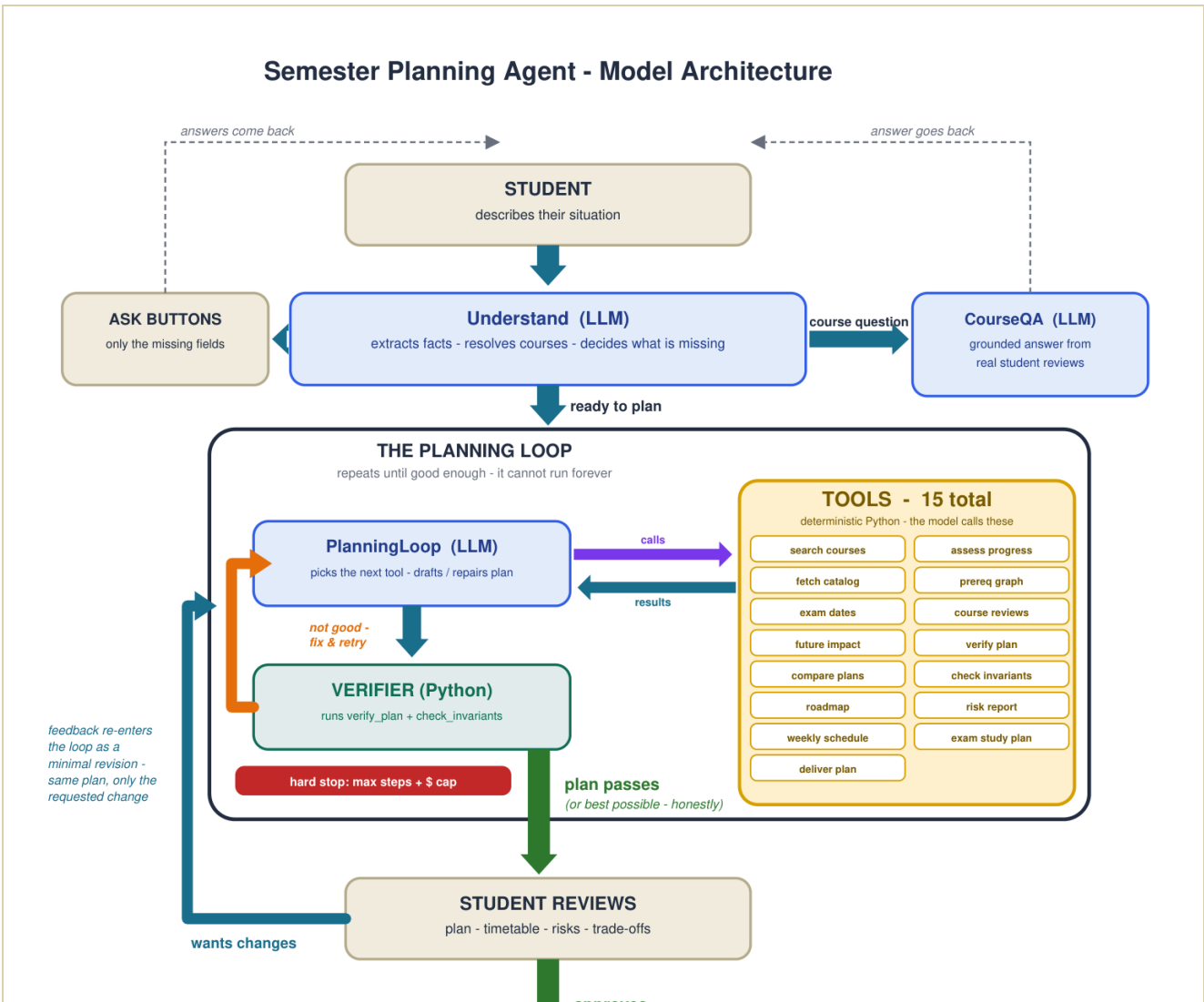


Everything We Built

A complete, grouped record of the system as submitted: the core agent architecture, the data it runs on, every student-facing feature, and every correctness bug found and fixed through live testing. Ordered by category, not by date — chronology lives in git history.

3	3	15	116+	~25
degree tracks	real data sources	model-callable tools	mocked checks	bugs fixed live





*Two loops drive the agent: the inner plan-verify loop, and the outer student-feedback loop.
The model is trusted with judgment - never with hard correctness.*

Blue = LLM modules. They appear under these exact names in the `/api/execute` steps trace:
Understand - PlanningLoop - CourseQA

Yellow = the 15 TOOLS (deterministic Python) the PlanningLoop model chooses from at every step.
3 diagnostics auto-run each turn (assess progress, fetch catalog, prereq graph); deliver_plan ends the turn.

How the agent works, end to end. A student's message is first read by **Understand** (an LLM call) which extracts structured facts — semester, passed/failed courses, constraints — and decides whether anything essential is missing. Once ready, the **PlanningLoop** takes over: an LLM-driven loop that freely calls any of 15 deterministic Python tools (yellow) — checking prerequisites, exam dates, schedules, degree progress — drafting and repairing a candidate plan until a **Verifier** (pure Python, green) confirms it satisfies every hard rule, or a step/cost ceiling is hit and the best achievable plan is delivered honestly. A pure course question routes instead to **CourseQA**, answered directly from real student reviews with no plan touched. The student's reply then re-enters the same loop as a minimal revision, never a from-scratch replan. These three module names appear verbatim in the `/api/execute` steps trace, matching this diagram exactly.

CONTENTS

- | | |
|----------------------------|---|
| 1. Core agent architecture | 4. Correctness fixes from live testing |
| 2. Data & tracks | 5. Design & UI |
| 3. Student-facing features | 6. Documentation & submission readiness |

1. Core agent architecture

The part that makes this an agent, not a script: the model chooses its own next step inside a bounded loop, with Python as the final word on correctness.

Item	Description
Bounded tool-calling loop	The model picks which of 15 tools to call and when it is confident enough to call <code>deliver_plan</code> — verified live via genuinely different tool sequences and step counts per request, not a fixed script.
One-shot pushback → code enforcement	Every hard rule (verify before deliver, no padding, an explicit removal/add/retake honored) gets one

	nudge back to the model, then Python enforces it directly on the second violation — never trusts the model's own bookkeeping.
Per-turn budget cap + step limit	Fixed the original per-conversation cap that killed long chats by turn 3; hard step ceiling so a turn can never loop forever.

2. Data & tracks

Item	Description
Technion SAP/OData API	Full catalog, prerequisite trees, schedules, exam dates — fetched once offline per track, never live during a request.
CheeseFork	Crowd difficulty/satisfaction ratings and real review excerpts, quoted directly.
technion-histograms	Real historical grade distributions — the comparison baseline behind grade-improvement retakes.
3rd track added NEW	הנדסת תעשייה וניהול, previously excluded entirely since Technion's tree has no flat mandatory list for it — digitizing its official diagram gave it real semester-by-semester data, now fully selectable.
Diagrams digitized, all 3 tracks FIX	Real per-course semester placement, hand-digitized and cross-checked credit-total-for-credit-total against each track's printed diagram — overrides the prerequisite-depth heuristic wherever known.
Tree over/under-inclusion FIX	Confirmed live, both directions: Technion's tree is missing real mandatory courses the diagrams show, and separately tags some courses "Mandatory" the diagrams never list — per-track, per-course corrections added, each verified against the real diagram.
Pipeline retry bug FIX	The fetch pipeline retried an empty response forever — the API's actual answer for "not offered this semester," a real permanent result, not a transient failure. Fixed to fail fast

3. Student-facing features

Item	Description
Transcript PDF upload	Parses an official Technion grade sheet directly into passed/failed courses and grades — replaces the least reliable step (free-text guessing) with ground truth. Never stores the raw PDF or its text anywhere.
Cross-session persistence	Supabase-backed, keyed by a hashed student email — restore a saved plan on a different browser or device, no password, no login wall. Requires a real <code>@campus.technion.ac.il</code> address.
Agentic grade-improvement retakes	Reworked from a fixed threshold into a genuine propose-then-approve loop: the agent compares a known grade against both the course average and the student's own average, proposes, and only includes it once the student agrees.
Retake persistence FIX	An accepted retake was previously protected for exactly one turn by design — any later, unrelated revision let the invariant checker silently strip it back out. Now persists for the rest of the conversation, the same way passed/failed courses already do.
Retake question uses the real grade FIX	"Do you recommend retaking X?" routed through a different code path than the planning loop, one that never looked at the student's known grade at all — fixed to share the same two-baseline comparison.
Explicit add-course negotiation	Not offered this semester → explained plainly, never forceable. A genuine schedule or credit conflict → named specifically, with a direct question asking whether to force it in anyway, trade-off disclosed as an open issue.

"Add X" when already passed FIX	Fell between the add-negotiation and retake mechanisms entirely — neither fired, nothing explained it. Now names the course, states it is already passed, and offers a concrete retake path.
Semester-1 completeness guarantee NEW	A first semester with no specific request defaults to exactly the track's real required courses — no sport or elective padding — and is topped up automatically if the model's own draft is incomplete.
Back to previous plan	Full plan history, not just a course-number diff — restores a prior plan completely (print, calendar export, and share image all keep working on it) and rewinds the conversation's memory to match.

4. Correctness fixes from live testing

Every item below was reproduced directly before being fixed, and shipped with a new permanent regression test — never guessed.

Bug	Fix
Workload formula scored an ordinary mandatory-heavy semester as "too heavy," so the model kept dropping the hardest — often mandatory — courses to bring the score down.	Recalibrated around a measured difficulty baseline instead of scaling from zero.
Technion's shared requirement tree paired a track's real required course with a different track's variant of the same subject under a fake "pick one" node — sometimes delivered the wrong course entirely.	A choice is only ever surfaced when neither option is already a confirmed real requirement.
Elective menu was effectively static — the same one or two courses suggested every time for a given scenario.	Shuffled, both in the model's own menu and a separate backstop that had bypassed it entirely.
A requested removal could come back with the course still in the plan.	Enforced in code on the second violation, with hard exclusion from

	every later top-up backstop.
Delivered weeks could contain overlapping class-time sections, never checked anywhere.	Coordinated section chooser plus a hard strip — a clashing week is never delivered.
"Meets on Sunday" was checked at the course level rather than per section.	Checking per-section removed roughly half of false schedule conflicts.
"Sundays are fine now" (relaxing a constraint) was silently ignored.	The state merge now distinguishes "not mentioned" from "explicitly cleared."
Hung network calls and rate-limit errors could freeze or crash a turn.	90-second per-call timeout; automatic retries with backoff.
A plan could ship with many open issues with no guard reacting.	Issue-budget guard: at most 2 open issues, none of them "lazy" ones, or the model is sent back.
A prerequisite-locked course could be delivered as a knowing "anchor."	Locked courses are stripped in code on every delivery path, with the specific missing prerequisite named honestly.
A crash in the explicit-add hard-enforcement path (an incorrect list mutation inside a filter) — would 500 the whole request the instant it fired, indistinguishable from the agent silently ignoring the request.	Fixed the mutation; every similar code path audited and confirmed safe.

5. Design & UI

Item	Description
Visual identity	Warm cream light background with a deep emerald accent. Dark mode rebuilt as warm espresso, not navy, so both themes read as one considered design.
Typography	Standardized on Open Sans throughout the application for clarity and consistency.

Full color-token sweep **FIX**

Every hardcoded color living outside the CSS token system — the confetti animation, the canvas-drawn share-image export, the favicon, the timetable's course-color palette — synced by hand and verified.

6. Documentation & submission readiness

Item	Description
LLMod key confirmed	Both local and the Vercel production environment switched back to the shared course key — verified with a full live regression pass after the switch, on both.
README, TESTING.md, checklist updated	New data-sources section, retake scenarios, add-request negotiation documentation, 5 new test scenarios, corrected track and test counts.
Project report rebuilt	The prior version predated almost everything above. Rebuilt to reflect the system as actually submitted.
Superseded handoff guide retired	Named a different submission URL/repository than what is actually being submitted; every other point in it is now covered by the documents above.

